

Pontifícia Universidade Católica de Goiás

Escola Politécnica e de Artes
Curso de Ciência da Computação

Atividade Final

Paradigmas de Linguagens de Programação
Sistema de Análise de Desempenho Acadêmico

Alunos:

Antônio Calixto da Silva
Amanda Barbosa Ferreira Agapito
Guilherme José da Silva
Matheus Silva Pains

Professor: Roussian de Ramos Alves Gaioso

Goiânia, Goiás
12 de Junho de 2026

Sumário

1	Descrição do Problema	2
1.1	Dados de Entrada	2
1.2	Resultado Esperado	2
2	Linguagens Escolhidas	2
3	Parte 1 — Implementação Imperativa (C)	3
3.1	Saída da Execução	4
4	Parte 2 — Implementação Orientada a Objetos (Python)	4
4.1	Saída da Execução	6
5	Parte 3 — Implementação Funcional (Python)	6
5.1	Saída da Execução	8
6	Parte 4 — Implementação Lógica (Prolog)	8
6.1	Exemplo de Consultas e Saída	9
7	Comparação de Legibilidade	9
8	Comparação de Facilidade de Escrita	10
9	Comparação de Confiabilidade	11
10	Comparação sobre Nomes, Escopo e Tempo de Vida	11
10.1	Versão Imperativa	11
10.2	Versão Orientada a Objetos	11
10.3	Versão Funcional	12
10.4	Versão Lógica (Prolog)	12
11	Comparação sobre Tipos de Dados	12
12	Comparação entre os Paradigmas	12
13	Conclusão	13

1 Descrição do Problema

O sistema recebe dados de alunos contendo nome, matrícula, três notas e frequência. A média é calculada como:

$$\text{média} = \frac{\text{nota1} + \text{nota2} + \text{nota3}}{3}$$

O aluno é **aprovado** se média $\geq 6,0$ e frequência $\geq 75\%$. Caso contrário, é **reprovado**.

1.1 Dados de Entrada

```
Ana;2024001;8.0;7.5;9.0;85
Bruno;2024002;5.0;6.0;4.5;80
Carlos;2024003;7.0;6.5;8.0;60
Daniela;2024004;9.0;9.5;10.0;95
Amanda;2024005;10;9;10;100
Guilherme;2024006;9;10;10;100
Matheus;2024007;10;10;9;100
```

1.2 Resultado Esperado

Nome	Matrícula	N1	N2	N3	Média	Freq	Situação
Ana	2024001	8,0	7,5	9,0	8,17	85	Aprovado
Bruno	2024002	5,0	6,0	4,5	5,17	80	Reprovado
Carlos	2024003	7,0	6,5	8,0	7,17	60	Reprovado
Daniela	2024004	9,0	9,5	10,0	9,50	95	Aprovado
Amanda	2024005	10,0	9,0	10,0	9,67	100	Aprovado
Guilherme	2024006	9,0	10,0	10,0	9,67	100	Aprovado
Matheus	2024007	10,0	10,0	9,0	9,67	100	Aprovado
Antonio	2024008	10,0	10,0	10,0	10,00	100	Aprovado

2 Linguagens Escolhidas

- C (imperativo)
- Python (orientado a objetos)
- Python (funcional)
- Prolog (lógico)

C foi escolhida para a versão imperativa por ser a linguagem sugerida pelo enunciado e a mais representativa do paradigma procedural. Python foi utilizada nas versões OO e funcional por sua versatilidade. Prolog foi mantido por ser a referência natural do paradigma lógico.

3 Parte 1 — Implementação Imperativa (C)

```
1  /* Parte 1 - Versao Imperativa em C */
2
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  int main() {
8      FILE *arq;
9      char linha[200];
10     char nome[50], mat[20];
11     float n1, n2, n3, media;
12     int freq, aprovados = 0, reprovados = 0, total = 0;
13
14     /* ---- 1. Pre-processamento (entrada de dados) ---- */
15
16     arq = fopen("alunos.csv", "r");
17     if (arq == NULL) {
18         printf("Erro ao abrir arquivo\n");
19         return 1;
20     }
21
22     printf("Nome          Matricula      Media      Freq      Situacao\n");
23     printf("-----\n");
24
25     while (fgets(linha, 200, arq) != NULL) {
26         linha[strcspn(linha, "\r\n")] = 0;
27         if (strlen(linha) == 0) continue;
28
29         strcpy(nome, strtok(linha, ";"));
30         strcpy(mat, strtok(NULL, ";"));
31         n1 = atof(strtok(NULL, ";"));
32         n2 = atof(strtok(NULL, ";"));
33         n3 = atof(strtok(NULL, ";"));
34         freq = atoi(strtok(NULL, ";"));
35
36         /* ---- 2. Processamento (calcula e classificacao) ---- */
37
38         media = (n1 + n2 + n3) / 3;
39
40         if (media >= 6.0 && freq >= 75) {
41             printf("%-12s %-12s %-7.2f %-5d Aprovado\n", nome, mat,
42                 media, freq);
43             aprovados++;
44         } else {
45             printf("%-12s %-12s %-7.2f %-5d Reprovado\n", nome, mat,
46                 media, freq);
47             reprovados++;
48         }
49         total++;
50     }
51
52     fclose(arq);
```

```

52  /* ---- 3. Resultado (saida final) ---- */
53
54  printf("-----\n");
55  printf("Total: %d | Aprovados: %d | Reprovados: %d\n", total,
56        aprovados, reprovados);
57
58  return 0;
59 }

```

Listing 1: parte1_imperativo.c

3.1 Saída da Execução

Nome	Matricula	Media	Freq	Situacao
-----	-----	-----	-----	-----
Ana	2024001	8.17	85	Aprovado
Bruno	2024002	5.17	80	Reprovado
Carlos	2024003	7.17	60	Reprovado
Daniela	2024004	9.50	95	Aprovado
Amanda	2024005	9.67	100	Aprovado
Guilherme	2024006	9.67	100	Aprovado
Matheus	2024007	9.67	100	Aprovado
Antonio	2024008	10.00	100	Aprovado
-----	-----	-----	-----	-----
Total: 8 Aprovados: 6 Reprovados: 2				

4 Parte 2 — Implementação Orientada a Objetos (Python)

```

1  # Parte 2 - Versao Orientada a Objetos (Python)
2
3  import os
4
5
6  class Aluno:
7      def __init__(self, nome, matricula, nota1, nota2, nota3, frequencia)
8          :
9          self.nome = nome
10         self.matricula = matricula
11         self.nota1 = nota1
12         self.nota2 = nota2
13         self.nota3 = nota3
14         self.frequencia = frequencia
15
16     def calcular_media(self):
17         return (self.nota1 + self.nota2 + self.nota3) / 3
18
19     def verificar_aprovacao(self):
20         media = self.calcular_media()
21         if media >= 6.0 and self.frequencia >= 75:
22             return "Aprovado"
23         else:
24             return "Reprovado"
25

```

```

26 class Turma:
27     def __init__(self):
28         self.alunos = []
29
30     def carregar_arquivo(self, caminho):
31         arquivo = open(caminho, "r", encoding="utf-8")
32         for linha in arquivo:
33             linha = linha.strip()
34             if linha == "":
35                 continue
36             partes = linha.split(";")
37             aluno = Aluno(
38                 partes[0],
39                 partes[1],
40                 float(partes[2]),
41                 float(partes[3]),
42                 float(partes[4]),
43                 int(partes[5])
44             )
45             self.alunos.append(aluno)
46         arquivo.close()
47
48     def gerar_relatorio(self):
49         print("=" * 70)
50         print("RELATORIO DE DESEMPENHO ACADEMICO - VERSAO 00 (Python)")
51         print("=" * 70)
52         print(f"{'Nome':<12} {'Matricula':<12} {'N1':<6} {'N2':<6} {'N3':<6} {'Media':<8} {'Freq':<6} {'Situacao'}")
53         print("-" * 70)
54
55         aprovados = 0
56         reprovados = 0
57
58         for aluno in self.alunos:
59             media = aluno.calcular_media()
60             situacao = aluno.verificar_aprovacao()
61             print(f"{aluno.nome:<12} {aluno.matricula:<12} {aluno.nota1:<6.1f} {aluno.nota2:<6.1f} {aluno.nota3:<6.1f} {media:<8.2f} {aluno.frequencia:<6} {situacao}")
62
63             if situacao == "Aprovado":
64                 aprovados += 1
65             else:
66                 reprovados += 1
67
68         print("-" * 70)
69         print(f"Total de alunos: {len(self.alunos)}")
70         print(f"Aprovados: {aprovados}")
71         print(f"Reprovados: {reprovados}")
72         print("=" * 70)
73
74
75 # ---- 1. Pre-processamento (entrada de dados) ----
76
77 caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "
    alunos.csv")

```

```

78 turma = Turma()
79 turma.carregar_arquivo(caminho)
80
81 # ---- 2. Processamento (calculo e classificacao) ----
82 # Os metodos calcular_medio() e verificar_aprovacao() da classe Aluno
83 # sao chamados dentro de gerar_relatorio()
84
85 # ---- 3. Resultado (saida final) ----
86
87 turma.gerar_relatorio()

```

Listing 2: parte2_oo.py

4.1 Saída da Execução

```

=====
RELATORIO DE DESEMPENHO ACADEMICO - VERSAO 00 (Python)
=====
Nome           Matricula    N1      N2      N3      Media    Freq    Situacao
-----
Ana            2024001      8.0     7.5     9.0     8.17     85     Aprovado
Bruno          2024002      5.0     6.0     4.5     5.17     80     Reprovado
Carlos         2024003      7.0     6.5     8.0     7.17     60     Reprovado
Daniela        2024004      9.0     9.5     10.0    9.50     95     Aprovado
Amanda         2024005      10.0    9.0     10.0    9.67     100    Aprovado
Guilherme      2024006      9.0     10.0    10.0    9.67     100    Aprovado
Matheus        2024007      10.0    10.0    9.0     9.67     100    Aprovado
Antonio        2024008      10.0    10.0    10.0    10.00    100    Aprovado
-----
Total de alunos: 8
Aprovados: 6
Reprovados: 2
=====

```

5 Parte 3 — Implementação Funcional (Python)

```

1  # Parte 3 - Versao Funcional (Python)
2
3  import os
4  from functools import reduce
5
6
7  # Funcoes puras
8
9  def parse_linha(linha):
10     partes = linha.strip().split(";")
11     return {
12         "nome": partes[0],
13         "matricula": partes[1],
14         "nota1": float(partes[2]),
15         "nota2": float(partes[3]),
16         "nota3": float(partes[4]),
17         "frequencia": int(partes[5])
18     }
19

```

```

20
21 def calcular_media(aluno):
22     return (aluno["nota1"] + aluno["nota2"] + aluno["nota3"]) / 3
23
24
25 def verificar_aprovacao(aluno):
26     return calcular_media(aluno) >= 6.0 and aluno["frequencia"] >= 75
27
28
29 def gerar_resultado(aluno):
30     media = calcular_media(aluno)
31     situacao = "Aprovado" if verificar_aprovacao(aluno) else "Reprovado"
32     return {
33         "nome": aluno["nome"],
34         "matricula": aluno["matricula"],
35         "nota1": aluno["nota1"],
36         "nota2": aluno["nota2"],
37         "nota3": aluno["nota3"],
38         "frequencia": aluno["frequencia"],
39         "media": media,
40         "situacao": situacao
41     }
42
43
44 def contar_situacao(acc, r):
45     if r["situacao"] == "Aprovado":
46         return (acc[0] + 1, acc[1])
47     else:
48         return (acc[0], acc[1] + 1)
49
50
51 # ---- 1. Pre-processamento (entrada de dados) ----
52
53 caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "
54     alunos.csv")
55 arquivo = open(caminho, "r", encoding="utf-8")
56 linhas = arquivo.readlines()
57 arquivo.close()
58 alunos = list(map(parse_linha, filter(lambda l: l.strip() != "", linhas)
59     ))
60
61 # ---- 2. Processamento (calculo e classificacao) ----
62
63 resultados = list(map(gerar_resultado, alunos))
64 aprovados = list(filter(lambda r: r["situacao"] == "Aprovado",
65     resultados))
66 total_aprov, total_reprov = reduce(contar_situacao, resultados, (0, 0))
67
68 # ---- 3. Resultado (saida final) ----
69
70 print("=" * 70)
71 print("RELATORIO DE DESEMPENHO ACADEMICO - VERSAO FUNCIONAL (Python)")
72 print("=" * 70)
73 print(f"{'Nome':<12} {'Matricula':<12} {'N1':<6} {'N2':<6} {'N3':<6} {'
74     Media':<8} {'Freq':<6} {'Situacao'}")

```



```

72 print("-" * 70)
73
74 for r in resultados:
75     print(f"{r['nome']:<12} {r['matricula']:<12} {r['nota1']:<6.1f} {r['',
76         nota2']:<6.1f} {r['nota3']:<6.1f} {r['media']:<8.2f} {r['',
77         frequencia']:<6} {r['situacao']}]")
78
79 print("-" * 70)
80 print(f"Total de alunos: {len(resultados)}")
81 print(f"Aprovados: {total_aprov}")
82 print(f"Reprovados: {total_reprov}")
83 print("=" * 70)
84
85 print("\n--- Aprovados (via filter) ---")
86 for a in aprovados:
87     print(f"    {a['nome']} - Media: {a['media']:.2f}")

```

Listing 3: parte3_funcional.py

5.1 Saída da Execução

```

=====
RELATORIO DE DESEMPENHO ACADEMICO - VERSAO FUNCIONAL (Python)
=====
Nome           Matricula      N1      N2      N3      Media      Freq      Situacao
-----
Ana            2024001         8.0     7.5     9.0     8.17       85       Aprovado
Bruno          2024002         5.0     6.0     4.5     5.17       80       Reprovado
Carlos         2024003         7.0     6.5     8.0     7.17       60       Reprovado
Daniela       2024004         9.0     9.5    10.0     9.50       95       Aprovado
Amanda        2024005        10.0     9.0    10.0     9.67      100       Aprovado
Guilherme     2024006         9.0    10.0    10.0     9.67      100       Aprovado
Matheus       2024007        10.0    10.0    10.0     9.67      100       Aprovado
Antonio       2024008        10.0    10.0    10.0    10.00      100       Aprovado
-----
Total de alunos: 8
Aprovados: 6
Reprovados: 2
=====

--- Aprovados (via filter) ---
Ana - Media: 8.17
Daniela - Media: 9.50
Amanda - Media: 9.67
Guilherme - Media: 9.67
Matheus - Media: 9.67
Antonio - Media: 10.00

```

6 Parte 4 — Implementação Lógica (Prolog)

```

1  % Parte 4 - Versao Logica (Prolog)
2
3  % ---- 1. Pre-processamento (entrada de dados) ----
4  % Fatos: aluno(Nome, Matricula, Nota1, Nota2, Nota3, Frequencia)
5
6  aluno(ana,          2024001, 8.0, 7.5, 9.0, 85).
7  aluno(bruno,        2024002, 5.0, 6.0, 4.5, 80).

```

```

8 aluno(carlos,      2024003, 7.0, 6.5, 8.0, 60).
9 aluno(daniela,     2024004, 9.0, 9.5, 10.0, 95).
10 aluno(amanda,      2024005, 10.0, 9.0, 10.0, 100).
11 aluno(guilherme,   2024006, 9.0, 10.0, 10.0, 100).
12 aluno(matheus,     2024007, 10.0, 10.0, 9.0, 100).
13 aluno(antonio,     2024008, 10.0, 10.0, 10.0, 100).
14
15 % ---- 2. Processamento (regras de calculo e classificacao) ----
16
17 media(Nome, Media) :-
18     aluno(Nome, _, N1, N2, N3, _),
19     Media is (N1 + N2 + N3) / 3.
20
21 frequencia_ok(Nome) :-
22     aluno(Nome, _, _, _, _, Freq),
23     Freq >= 75.
24
25 aprovado(Nome) :-
26     media(Nome, Media),
27     Media >= 6.0,
28     frequencia_ok(Nome).
29
30 reprovado(Nome) :-
31     aluno(Nome, _, _, _, _, _),
32     \+ aprovado(Nome).
33
34 % ---- 3. Resultado (consultas para saida) ----
35 % Exemplos de uso:
36 % ?- aprovado(X).
37 % ?- reprovado(X).
38 % ?- media(ana, M).

```

Listing 4: parte4_logico.pl

6.1 Exemplo de Consultas e Saída

```

?- aprovado(X).
X = ana ;
X = daniela ;
X = amanda ;
X = guilherme ;
X = matheus ;
X = antonio.

?- reprovado(X).
X = bruno ;
X = carlos.

?- media(ana, M).
M = 8.166666666666666.

```

7 Comparação de Legibilidade

Conclusão: Para leitores iniciantes, a versão imperativa é a mais legível. Para projetos maiores, a OO oferece melhor organização. Prolog é o mais conciso, mas exige familiari-

Critério	Análise
Clareza dos nomes	Todas as versões Python usam nomes descritivos em português. Prolog usa átomos curtos (ana , aprovado), concisos mas claros.
Organização	A versão OO é a mais organizada: dados e lógica ficam encapsulados em classes. A funcional separa bem entrada/processamento/saída. A imperativa é linear e direta.
Detalhes sintáticos	Prolog tem a menor quantidade de sintaxe. Python imperativo exige mais linhas por usar while e índices explícitos.
Fluxo do programa	A imperativa é a mais fácil de seguir passo a passo. A funcional exige entender composição de funções. Prolog não tem fluxo explícito.
Localização das regras	Em Prolog, cada regra é uma cláusula isolada e fácil de encontrar. Na OO, estão nos métodos da classe. Na funcional, em funções nomeadas.

Tabela 1: Comparação de legibilidade entre os paradigmas.

dade com o paradigma.

8 Comparação de Facilidade de Escrita

Critério	Análise
Quantidade de código	Prolog: ~40 linhas. Funcional: ~70 linhas. Imperativa: ~80 linhas. OO: ~100 linhas.
Dificuldade sintática	Python é uniforme nos três paradigmas. Prolog exige aprender unificação e backtracking.
Manipulação de listas	A funcional é a mais natural com map/filter . A imperativa usa índices manuais. A OO itera com for .
Estruturas auxiliares	A OO exige criar classes. A funcional usa dicionários. A imperativa usa listas de listas. Prolog usa fatos diretamente.
Bibliotecas	Apenas a funcional importa functools.reduce . As demais usam apenas a biblioteca padrão.

Tabela 2: Comparação de facilidade de escrita.

Conclusão: A versão funcional Python foi a mais rápida de implementar pela expressividade de **map/filter**. Prolog é extremamente conciso, mas exige mudança de mentalidade.

9 Comparação de Confiabilidade

Critério	Análise
Verificação de tipos	Python tem tipagem dinâmica — erros de tipo só aparecem em tempo de execução. Prolog não tem tipos explícitos.
Erros em tempo de execução	A imperativa é a mais suscetível a erros de índice (<code>alunos[i][5]</code>). A OO protege dados com encapsulamento. A funcional, ao usar funções puras, reduz efeitos colaterais.
Entrada inválida	Nenhuma versão trata entradas inválidas de forma robusta. Seria necessário adicionar <code>try/except</code> em Python ou validação em Prolog.
Acesso à memória	Python e Prolog não permitem acesso direto à memória, eliminando riscos de buffer overflow (diferente de C).
Testabilidade	A funcional é a mais testável: funções puras sem efeito colateral podem ser testadas isoladamente. A OO permite testes unitários por método.

Tabela 3: Comparação de confiabilidade.

Conclusão: A versão funcional oferece maior confiabilidade por usar funções puras. A OO vem em segundo lugar pelo encapsulamento. Python e Prolog são mais seguros que C por não permitirem acesso direto à memória.

10 Comparação sobre Nomes, Escopo e Tempo de Vida

10.1 Versão Imperativa

- **Nomes declarados:** `caminho_arquivo`, `lista_alunos` (globais); `nome`, `media`, `i` (locais em funções).
- **Visibilidade:** As variáveis globais são visíveis em todo o módulo. As locais existem apenas dentro de suas funções.
- **Escopo:** Funções como `calcular_media()` têm escopo local; parâmetros e variáveis internas não vazam.
- **Tempo de vida:** Variáveis locais são criadas na chamada da função e destruídas ao retornar. As globais existem durante toda a execução.

10.2 Versão Orientada a Objetos

- **Nomes declarados:** Classes `Aluno`, `Turma`; atributos `self.__nome`, `self.__nota1`; métodos `calcular_media()`.

- **Visibilidade:** Atributos com `__` (name mangling) são visíveis apenas dentro da classe. Métodos públicos são acessíveis externamente.
- **Escopo:** Cada objeto tem seu próprio escopo de atributos. Métodos acessam `self` para dados da instância.
- **Tempo de vida:** Objetos existem enquanto houver referência a eles. O garbage collector de Python os destrói quando não há mais referências.

10.3 Versão Funcional

- **Nomes declarados:** Funções puras `calcular_media()`, `verificar_aprovacao()`, `gerar_resultado()`; variável `alunos` no nível do módulo.
- **Visibilidade:** Cada função vê apenas seus parâmetros e variáveis locais. Não há estado mutável compartilhado.
- **Escopo:** Funções são de escopo global. Lambdas em `filter()` capturam variáveis do escopo envolvente (closure).
- **Tempo de vida:** Dados são criados a cada chamada e descartados após o retorno. Listas intermediárias (`resultados`) vivem no escopo do módulo.

10.4 Versão Lógica (Prolog)

- **Nomes declarados:** Predicados `aluno/6`, `media/2`, `aprovado/1`; variáveis lógicas `Nome`, `Media`.
- **Visibilidade:** Fatos e regras são globais no programa. Variáveis lógicas existem apenas dentro de sua cláusula.
- **Escopo:** Cada cláusula é independente. A variável `Nome` em `aprovado(Nome)` só existe naquela regra.
- **Tempo de vida:** Variáveis são instanciadas durante a unificação e deixam de existir quando a consulta termina ou faz backtracking.

11 Comparação sobre Tipos de Dados

12 Comparação entre os Paradigmas

1. **Como a versão imperativa organiza a solução?**
Sequencialmente: lê dados, percorre com laço `while`, calcula média, verifica condição, imprime. O programador controla cada passo.
2. **Como a versão orientada a objetos organiza a solução?**
Modela o domínio: `Aluno` encapsula dados e comportamento, `Turma` gerencia a coleção. A lógica é distribuída em métodos.

Aspecto	Python (3 versões)	Prolog
Tipagem	Dinâmica, forte	Sem tipos explícitos
Tipos primitivos	<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>	Átomos, números, variáveis
Strings	Tipo nativo <code>str</code> , imutável	Átomos (<code>ana</code> , <code>bruno</code>)
Coleções	Listas, dicionários, tuplas	Listas Prolog, termos compostos
Objetos	Sim (versão OO)	Não existe o conceito
Conversão de tipo	Explícita: <code>float()</code> , <code>int()</code>	Automática via <code>is</code>
Erros de tipo	<code>TypeError</code> em tempo de execução	Falha na unificação

Tabela 4: Comparação de tipos de dados.

3. Como a versão funcional organiza a solução?

Como pipeline de transformações: `ler_dados` $\rightarrow$ `map(gerar_resultado)` $\rightarrow$ `imprimir_relatorio`.
Dados fluem entre funções puras.

4. Como a versão declarativa organiza a solução?

Declara fatos (dados dos alunos) e regras (média, aprovação). O motor de inferência de Prolog encontra as respostas.

5. Qual versão ficou mais próxima da forma humana?

Prolog. A regra `aprovado(Nome) :- media(Nome, M), M >= 6.0, ...` é quase linguagem natural.

6. Qual versão ficou mais próxima da máquina?

A imperativa. O laço `while` com índice manual e atribuições explícitas reflete como o processador executa instruções.

7. Qual versão parece mais adequada para esse problema?

A funcional. O problema é essencialmente uma transformação de dados (entrada $\rightarrow$ processamento $\rightarrow$ saída), que é o caso de uso ideal do paradigma funcional.

13 Conclusão

Cada paradigma demonstrou forças e limitações claras ao resolver o mesmo problema:

- O **paradigma imperativo** é o mais intuitivo para iniciantes e oferece controle total sobre o fluxo, mas gera código mais verboso e propenso a erros de índice.
- O **paradigma orientado a objetos** brilha em organização e manutenibilidade. Encapsulamento protege os dados e facilita extensões futuras (ex.: adicionar novos tipos de aluno). É o mais adequado para sistemas grandes.
- O **paradigma funcional** produziu o código mais conciso e testável. Funções puras sem efeito colateral facilitam depuração e paralelismo. É ideal para problemas de transformação de dados.

- O **paradigma lógico** é o mais declarativo e conciso. O programador descreve *o que* quer, não *como* fazer. É poderoso para regras de negócio e sistemas especialistas, mas limitado para I/O e interfaces.

Não existe paradigma universalmente “melhor”. A escolha depende do problema: sistemas corporativos favorecem OO; pipelines de dados favorecem funcional; regras de inferência favorecem lógico; scripts simples favorecem imperativo. O programador competente domina múltiplos paradigmas e escolhe o mais adequado para cada situação.